BakeWish - Presentation & Demo Script

8-Minute Voice-Over Script formatted by Assignment Criteria

1. A Brief Introduction to the Selected Scenario and App Purpose

[ON-SCREEN ACTION]: Open the app on MainActivity (Home Screen). Scroll up and down slowly to show the header, welcome card, summary cards, and baking tips.

SPEAK (VOICE-OVER):
"Hello everyone! In this video, I am demonstrating my native Java Android application called BakeWish - Artisanal Patisserie.

The scenario for this application is a bakery wishlist tracker. Bakery lovers frequently want a simple, beautiful way to track treats they wish to try, along with prices, priority levels, and purchase status.

BakeWish solves this by allowing users to manage their bakery items, filter them by category, search through them in real time, and calculate total expenses in LKR. Visually, the app uses a custom Strawberry & Cream color theme with cream backgrounds, blush pink surface cards, and dark berry text."

2. Demonstration of the Application's Activities/Screens and Navigation

[ON-SCREEN ACTION]: Tap the bottom navigation menu items slowly: Home -> Add Item -> Wishlist -> Home.

SPEAK (VOICE-OVER):
"Let's explore the screens and navigation structure of the application.

The app consists of three main Activities:
1. MainActivity (Home Screen): Displays summary cards, recently added items, baking tips, and quick action buttons.
2. AddEditActivity (Add/Edit Screen): Contains form fields to add new bakery items or update existing ones.
3. WishlistActivity (My Wishlist): Displays all saved treats, real-time live search, purchase checkboxes, and dynamic statistics.

Users navigate between screens seamlessly using the bottom navigation bar. Notice how the active tab highlights with a dark berry rounded pill shape. The navigation bar uses WindowInsets handling in Java to elevate tab buttons safely above the Android bottom gesture bar."

3. Demonstration of All Four CRUD Operations (Create, Read, Update, Delete)

[ON-SCREEN ACTION]: Demonstrate each CRUD operation live on screen:
- CREATE: Click 'Add Bakery Item', fill form ('Strawberry Shortcake', Cake, 1250 LKR, High priority). Show category preview updating. Click Save Item.
- READ: Show newly added item on Home screen and Wishlist screen. Point out summary counts.
- UPDATE: Click edit icon on Strawberry Shortcake. Change price to 1350 LKR and click Update Item. Go to Wishlist and tap 'Purchased' checkbox.
- DELETE: Click red trash icon on Wishlist item. Show Toast message 'Item deleted from wishlist'.

SPEAK (VOICE-OVER):
"Now I will demonstrate all four CRUD operations and explain their underlying Java code:

1. CREATE (Add Item): In AddEditActivity, I fill in the item details. Selecting a category in the Spinner triggers an

OnItemSelectedListener that updates the preview image. Clicking 'Save Item' creates a new BakeryItem object and executes BakeryRepository.addItem(), which places the item at index 0 of an ArrayList in memory.

2. READ (View List & Totals): On MainActivity and WishlistActivity, BakeryRepository.getAllItems() fetches the list. Java loops through the items to calculate Total Count and Wishlist Value, and dynamically inflates item XML layouts into vertical card containers.

3. UPDATE (Edit & Checkbox): Clicking the edit button re-opens the form with pre-filled details. Updating the price executes BakeryRepository.updateItem(), which matches the unique item ID and updates its properties in memory. Additionally, tapping the 'Purchased' CheckBox updates purchase status and recalculates unpurchased values dynamically.

4. DELETE (Remove Item): Clicking the red delete icon triggers BakeryRepository.deleteItem(), which uses removeIf() to remove the item matching that ID from memory. A Toast message confirms the deletion and stats recalculate immediately."

4. An Explanation of How User Input is Processed and Output is Displayed

[ON-SCREEN ACTION]: Show input validation by attempting to save an empty form in AddEditActivity. Show error text and Toast. Then show search bar filtering in WishlistActivity.

SPEAK (VOICE-OVER):
"Let's look at how user inputs are processed and displayed:

1. Form Validation: When adding or editing an item, Java checks if the item name or price fields are empty. If invalid, the app displays a red error message and triggers a Toast notification.

2. Dynamic Category Image Selection: Selecting a category from the Spinner dropdown dynamically maps to the matching drawable resource (ic_cake, ic_donut, ic_bun, ic_pastry, ic_cookie) and updates ivCategoryPreview.

3. Real-Time Search Filtering: In WishlistActivity, typing into the search EditText triggers a TextWatcher listener that filters the list in real time by name or category.

4. Output Formatting: Prices are formatted in LKR using String.format(Locale.getDefault(), '%,.2f', price) for professional currency display."

5. An Explanation of the Android UI Components Used in the Application

[ON-SCREEN ACTION]: Slowly scroll through AddEditActivity and WishlistActivity while listing the UI components.

SPEAK (VOICE-OVER):
"Here are the Android UI components used in the application:

- ConstraintLayout: Used as the root layout on all screens for responsive view positioning.
- ScrollView: Wraps screen content so all forms and cards scroll smoothly on any phone screen.
- LinearLayout: Used for horizontal card rows and vertical product lists.
- TextView & EditText: Used for text labels, headings, prices, and user text/number inputs.
- Spinner: Used for the category dropdown selector.
- RadioGroup & RadioButton: Used for setting Low, Medium, or High priority levels.
- CheckBox: Used for tracking item purchase status.
- Button & ImageButton: Used for primary actions, navigation, edit, and delete triggers.
- Toast: Used for providing instant feedback to the user after actions.

- Custom Drawables & Shape Vector Icons: Used for rounded cards, pill buttons, category badges, and active navigation indicators."

6. A Brief Explanation of How the Application Satisfies Assignment Requirements

[ON-SCREEN ACTION]: Return to MainActivity showing the complete app interface.

SPEAK (VOICE-OVER):
"In conclusion, the BakeWish application satisfies all assignment criteria:

- Developed natively using Android Studio, Java, and XML layouts.
- Contains multiple Activities with smooth bottom navigation.
- Implements all four CRUD operations (Create, Read, Update, Delete) with in-memory data management.
- Includes robust user input validation and Toast feedback.
- Features a cohesive, professionally designed Strawberry & Cream UI/UX theme.

Thank you very much for watching my presentation!"